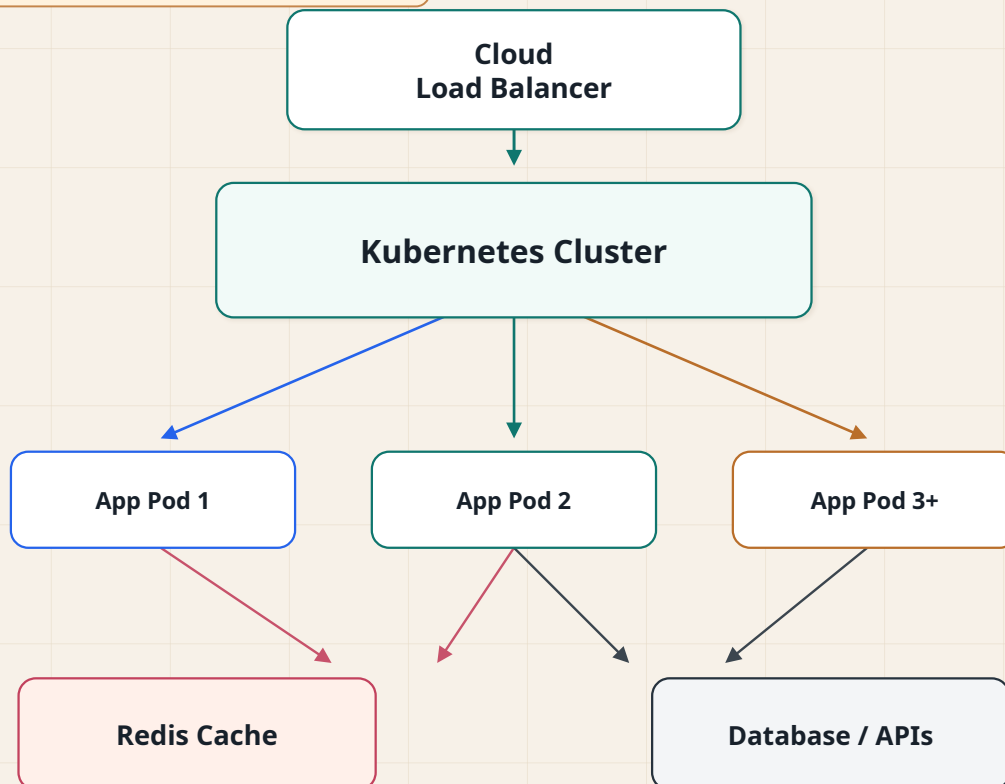


Scaling a Real Estate Platform with Kubernetes

Load Balancing - Horizontal Scaling - Redis Caching

How I redesigned infrastructure to support growing real estate traffic and API-heavy workloads.

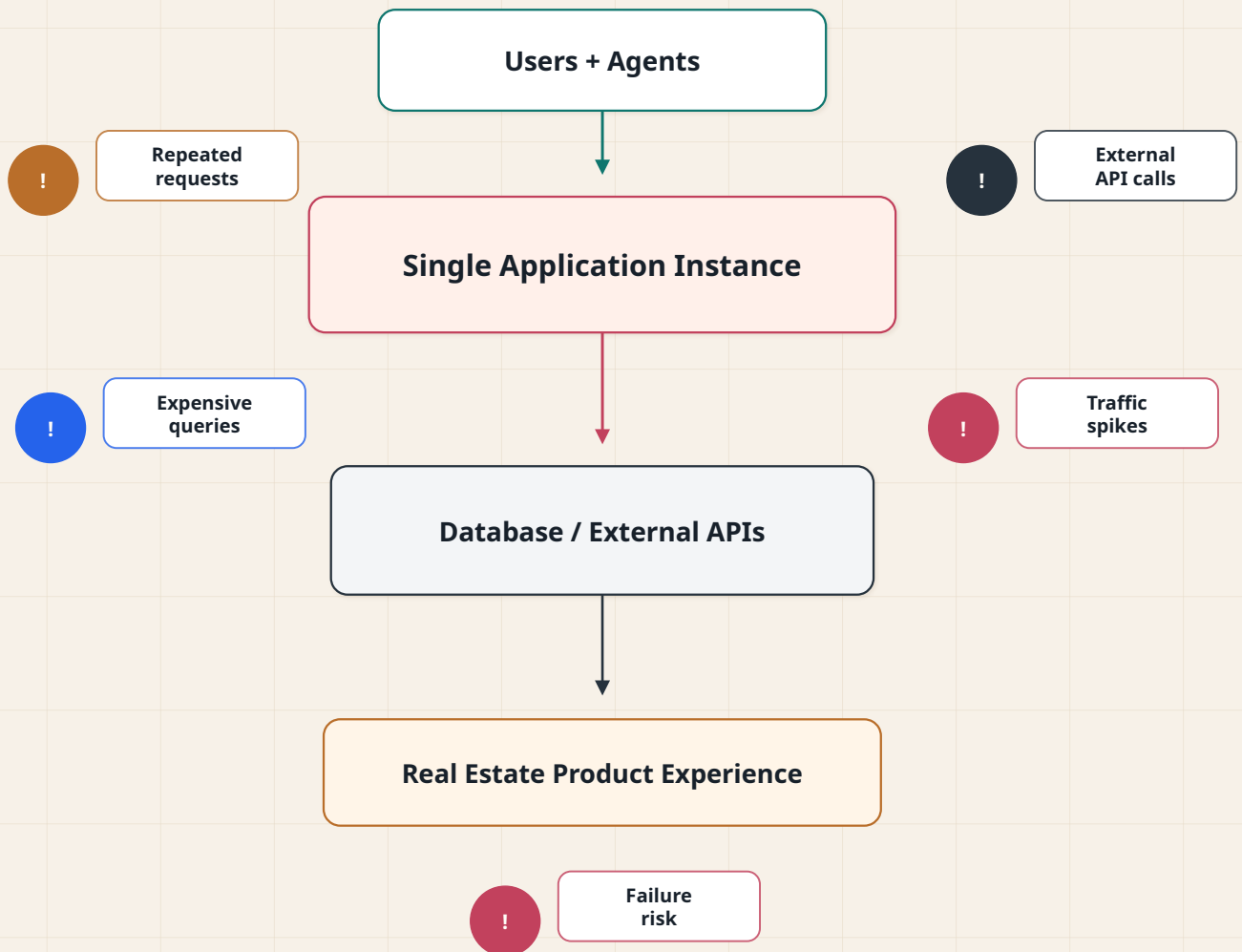
Case Study by Muhammad Moez Khan



Scalable real estate traffic + API-heavy workloads

Growth Created an Infrastructure Problem

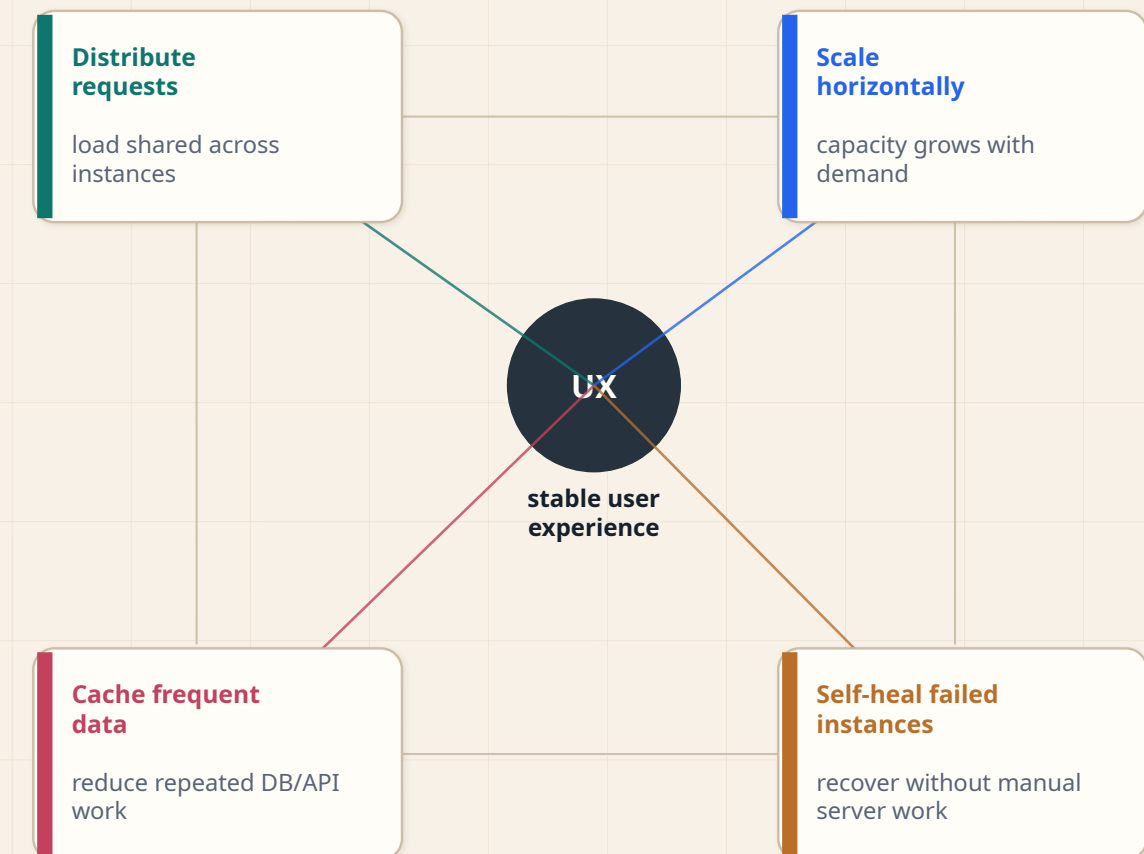
The platform served property inventory, listing pages, CRM integrations, dashboards and repeated API requests.



A growing product cannot depend on one application process doing all the work.

Build for Traffic, Failure and Growth

The goal was a platform that could scale capacity, protect data sources and recover from instance failure.

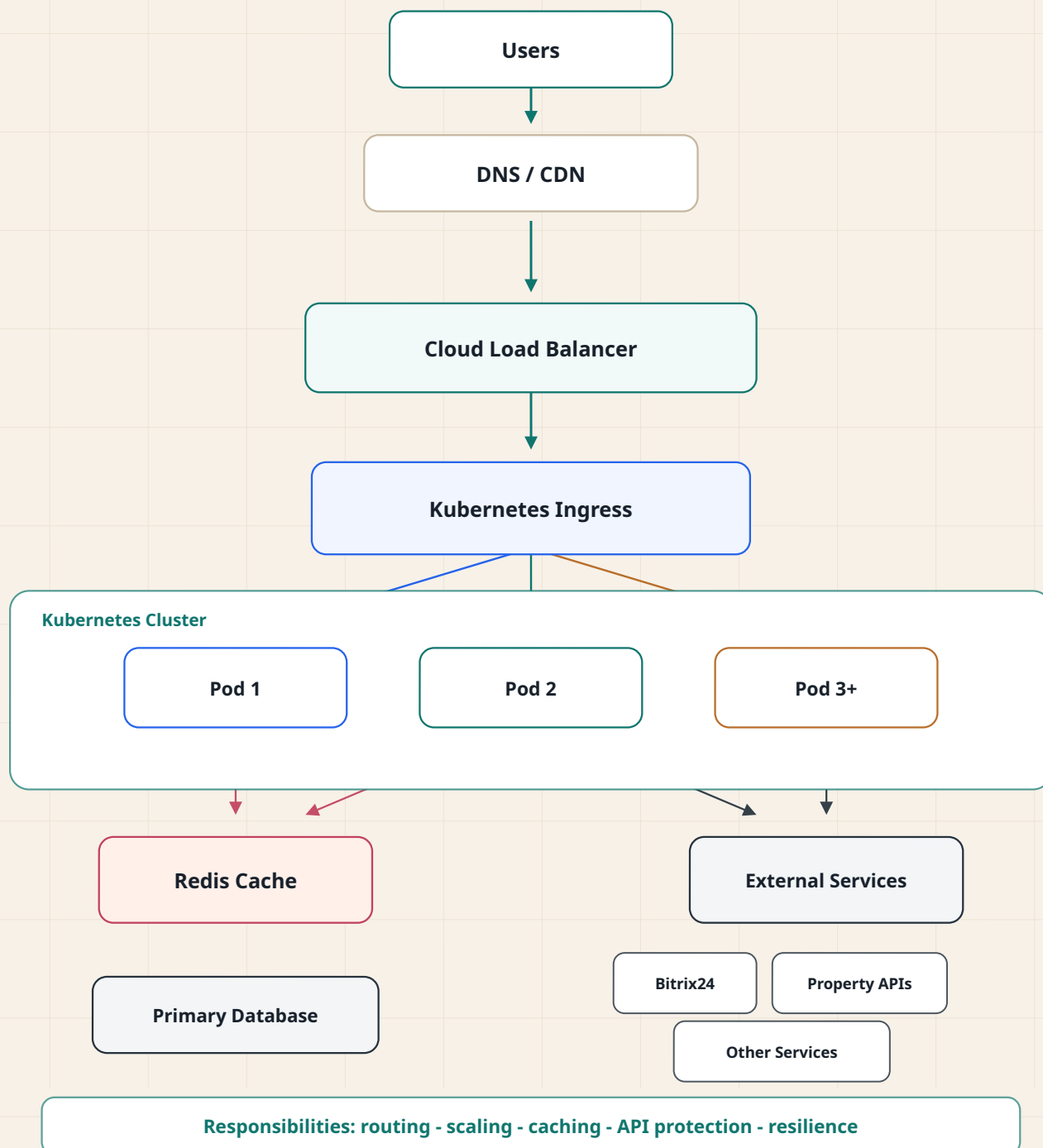


Prepare for growth: traffic spikes, larger property data and future workload increases

Scale the infrastructure without changing the business experience for users.

Production Architecture

A clear production path: balance traffic, route to pods, cache repeat work and keep sources authoritative.

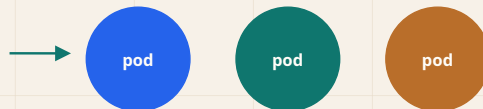


Scaling the Application, Not the Server

Instead of continuously increasing one machine, Kubernetes adds or removes application pods as demand changes.

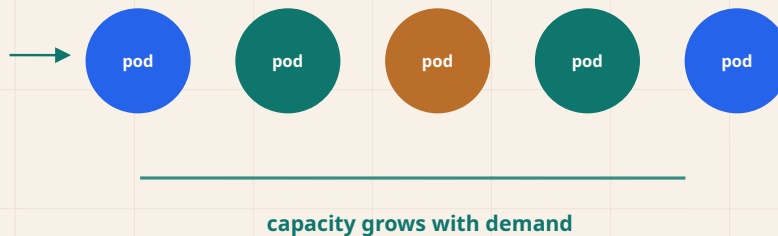
Normal traffic

3 pods



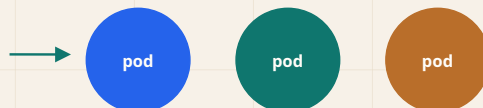
Traffic increase

3 -> 5 -> 8 pods



Traffic decrease

back to baseline



Autoscaling signals

CPU, memory and request/load metrics where configured

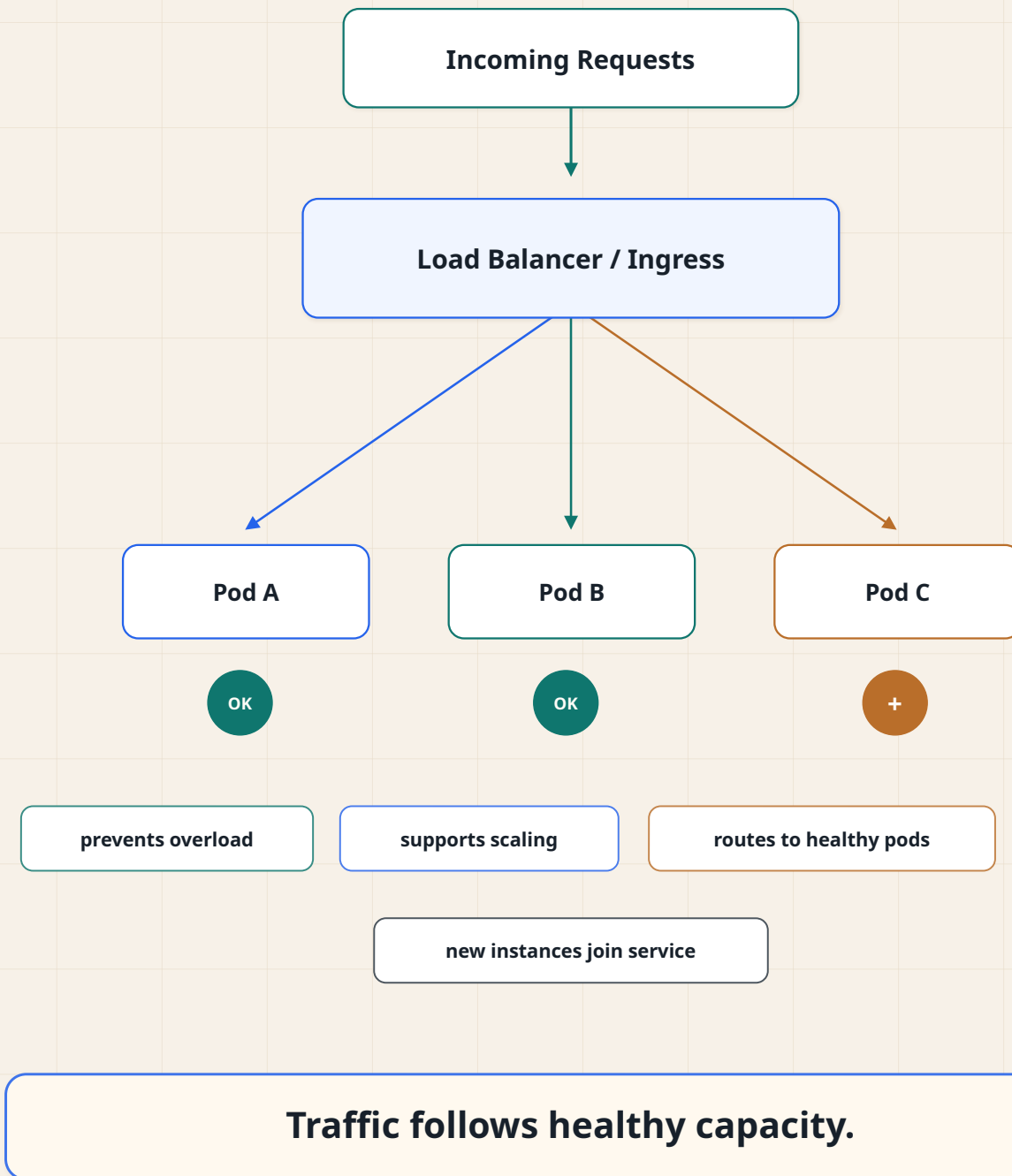
Self-healing

Unhealthy pods can be removed and replaced

Horizontal scaling allows capacity to grow with demand.

No Single Application Instance Handles Everything

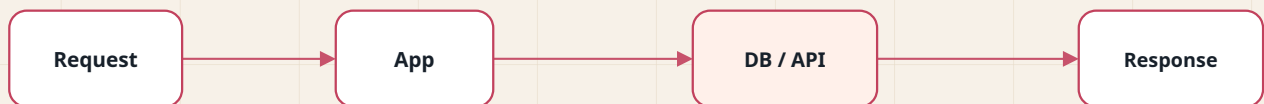
Load balancing turns multiple application pods into one reliable service endpoint.



Not Every Request Should Reach the Database

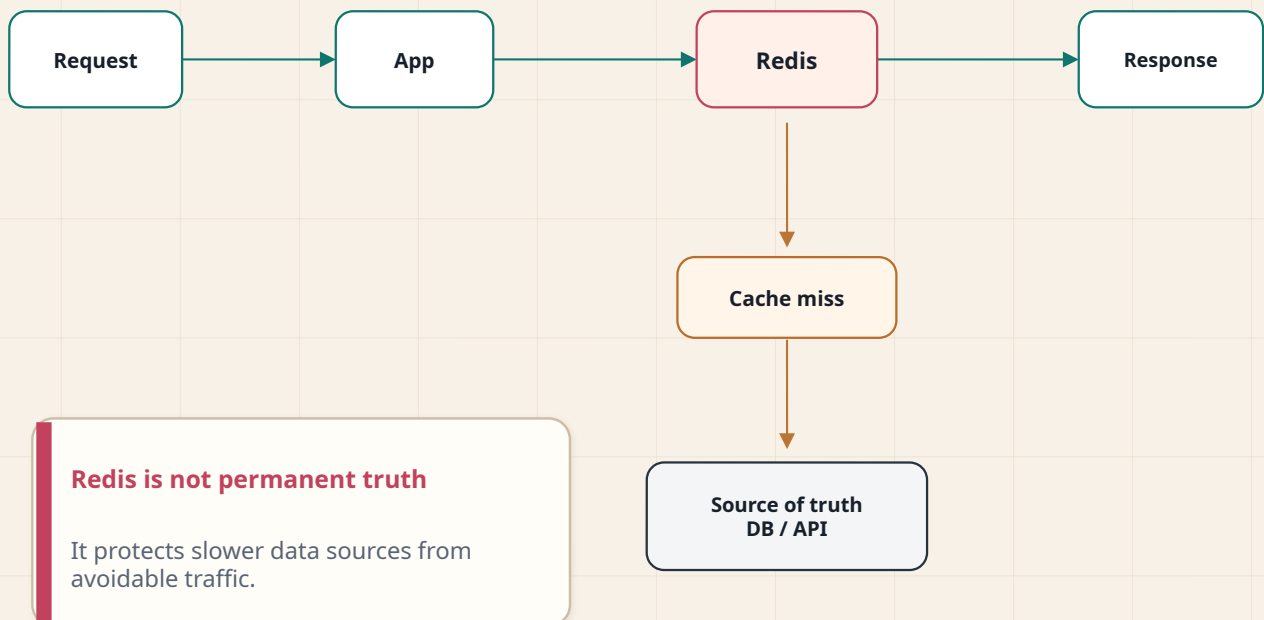
Redis reduces unnecessary repeated work for property details, search filters, dashboards and API responses.

WITHOUT CACHE



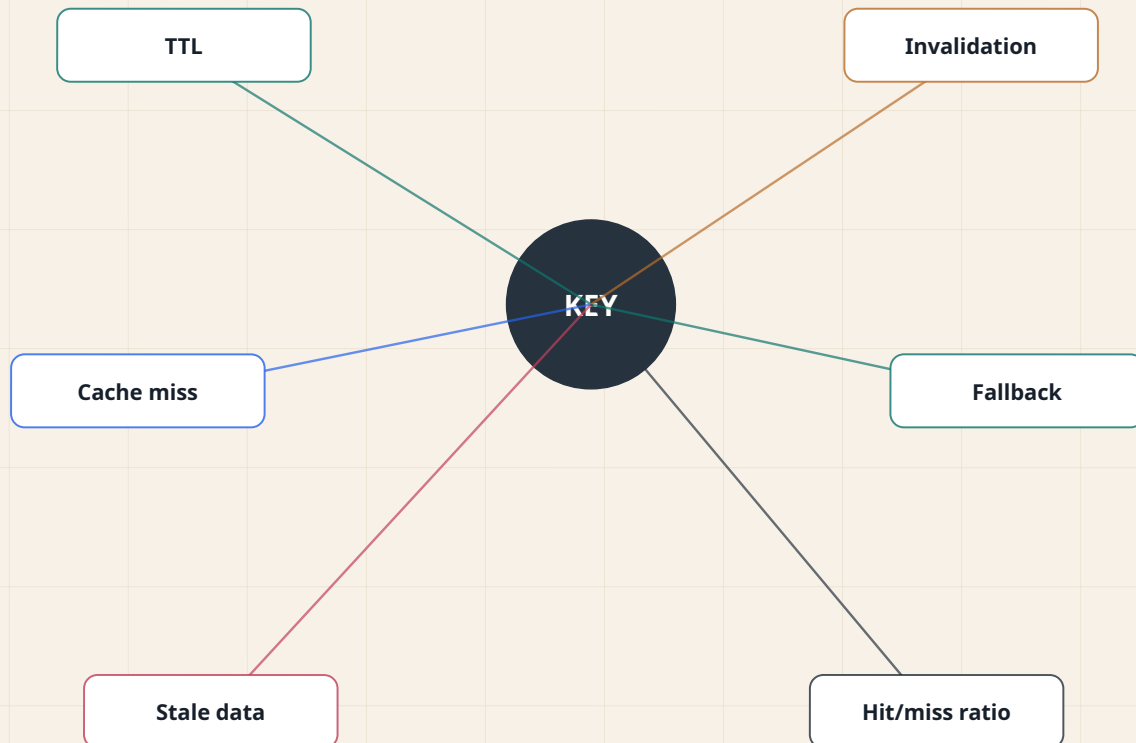
Every request repeats the same expensive path.

WITH REDIS



Caching Needs Rules, Not Just Redis

The engineering work is deciding what can be cached, how long it stays fresh and how it falls back.



Cache key examples

property:{id}

property-search:{filters}

dashboard:{department}:{period}

reference:{location}

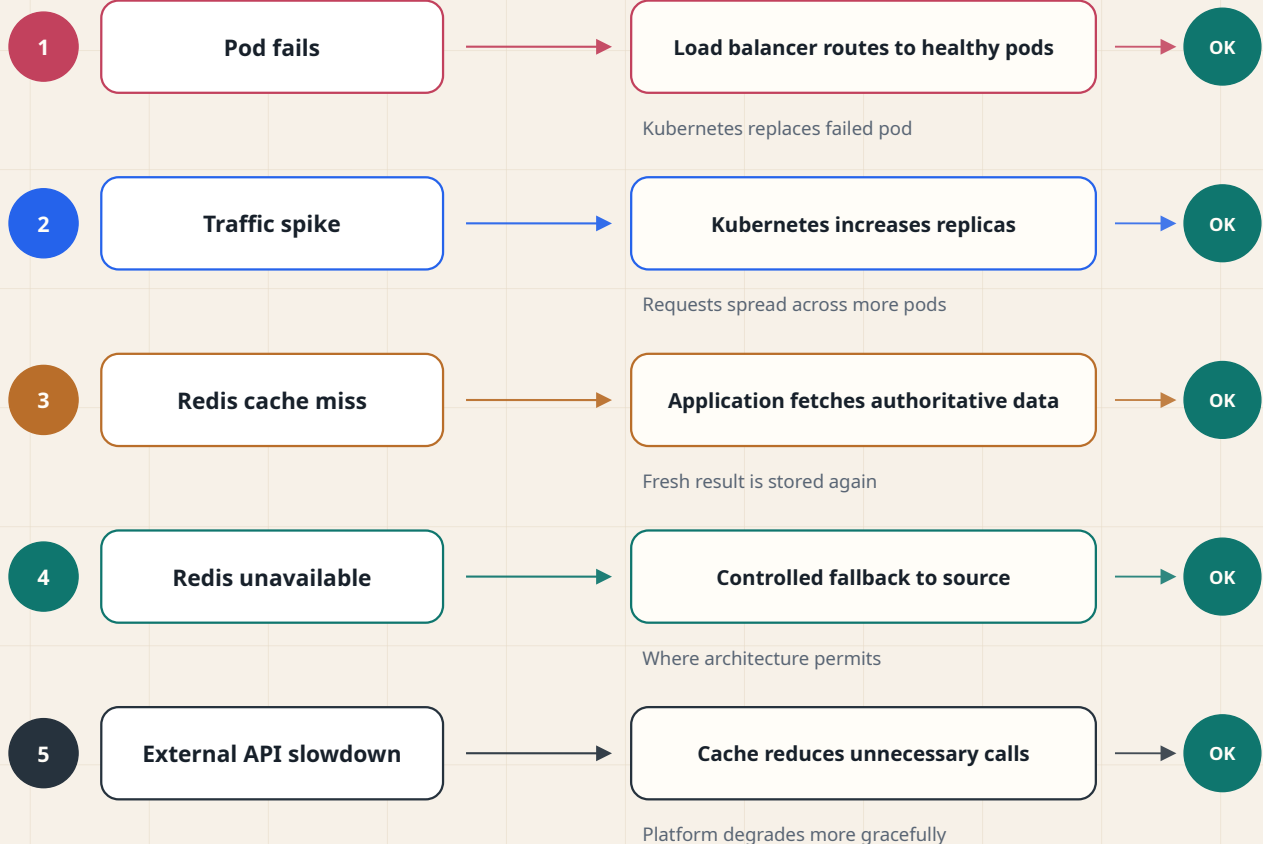
Fast data is useful only when its freshness rules are understood.

Designing for Failure

The system does not try to avoid every failure. It contains failures so they do not become outages.

Failure scenario

Controlled system response



Design principle: route away from unhealthy capacity, scale when pressure rises, and fall back to authoritative data when cache cannot serve.

Resilience comes from controlled behavior under stress.

What Infrastructure Engineering Changes

Platform decisions translate into reliability, traffic readiness and operational confidence.

BEFORE

Single server dependency

Repeated DB/API work

Traffic spikes create risk

Manual scaling pressure

Machine-bound infrastructure

AFTER

Multiple app instances

Load-balanced traffic

Redis-backed caching

Horizontal scalability

Improved fault tolerance

Business outcomes

More reliable
experience

Ready for traffic
spikes

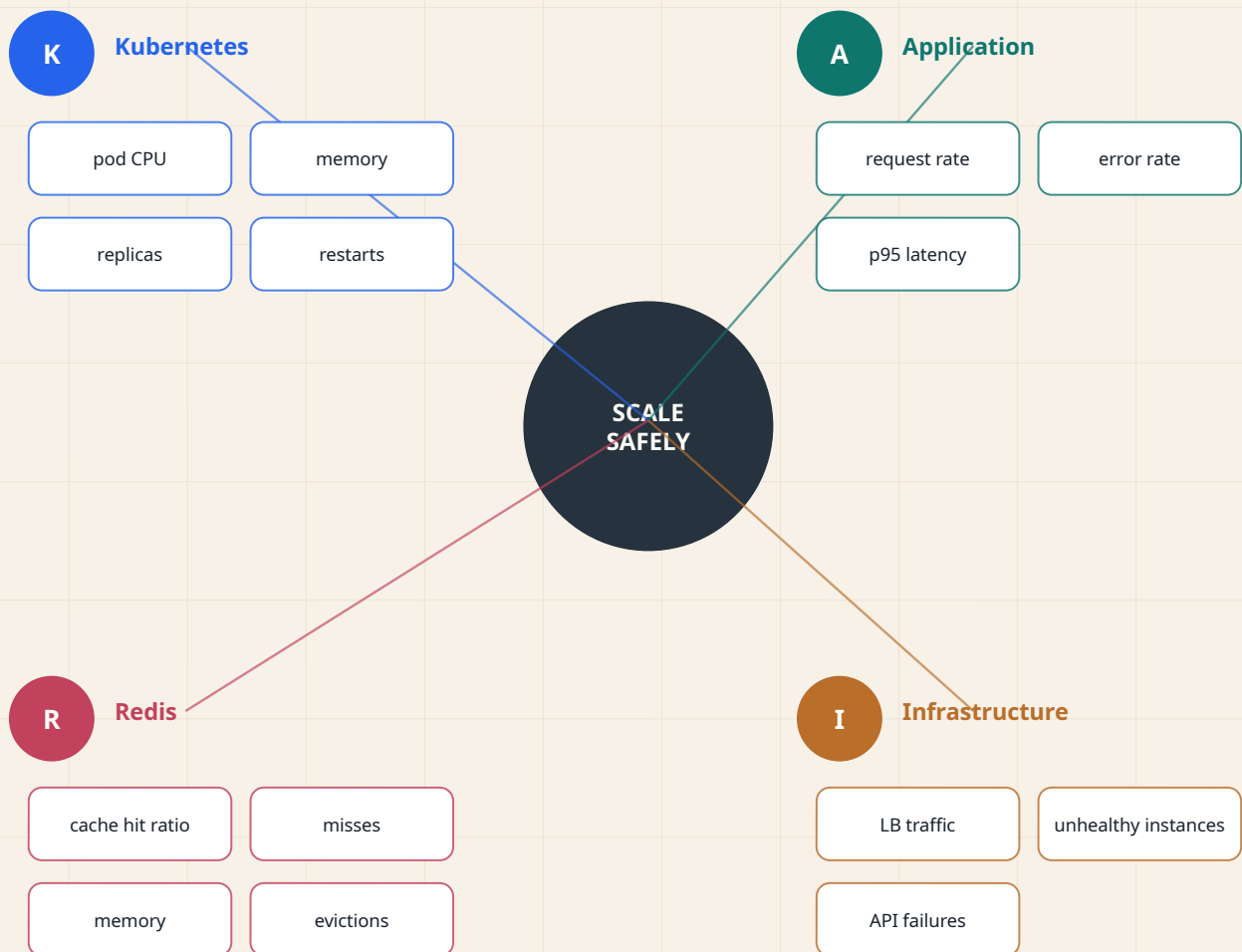
Lower pressure on
external APIs

Better operational
resilience

Metrics to add when available: p95 latency - cache hit ratio - API call reduction - error rate

Scaling Without Visibility Is Guessing

A scalable architecture still needs signals: pods, application health, cache behavior and infrastructure status.



Observe



Decide

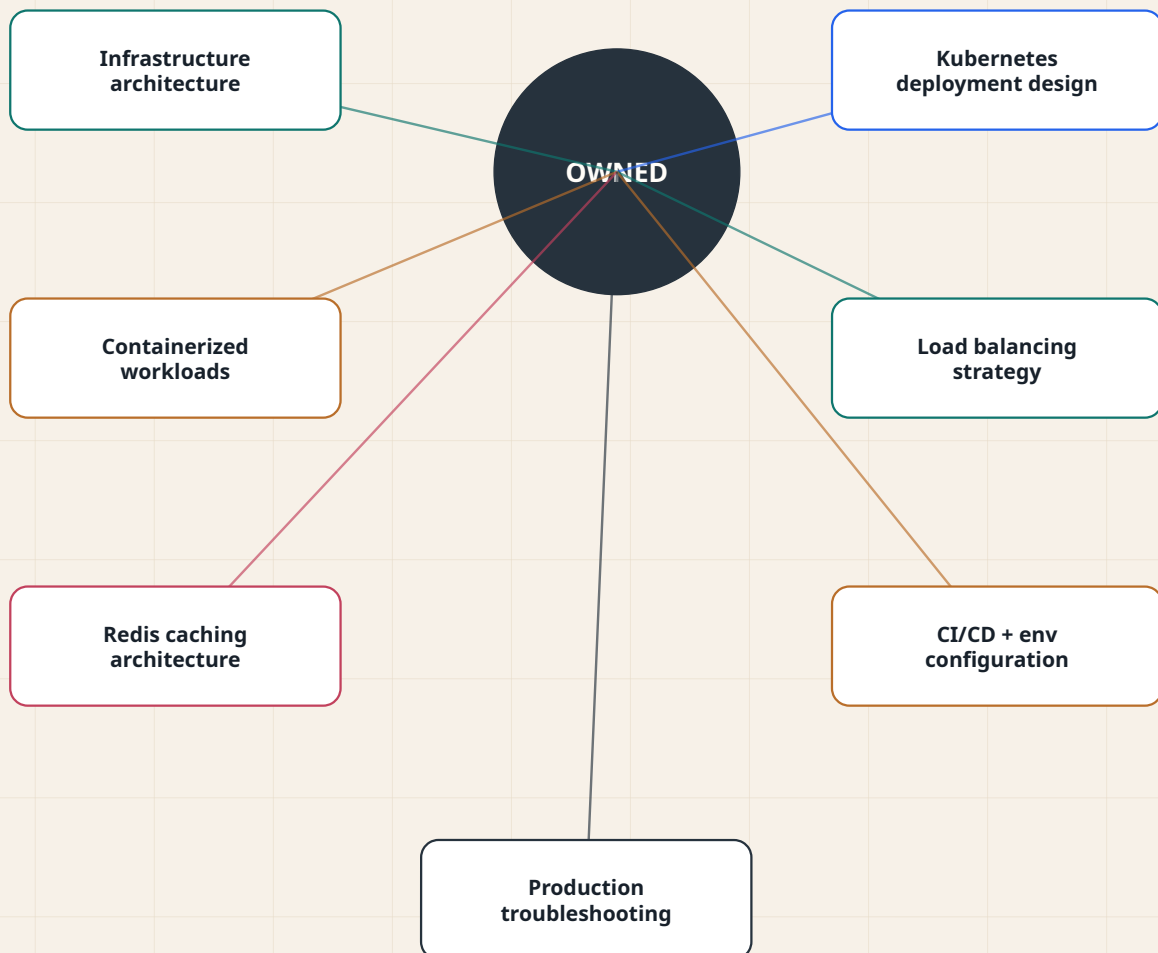


Scale

What I cannot measure, I cannot safely scale.

My Role

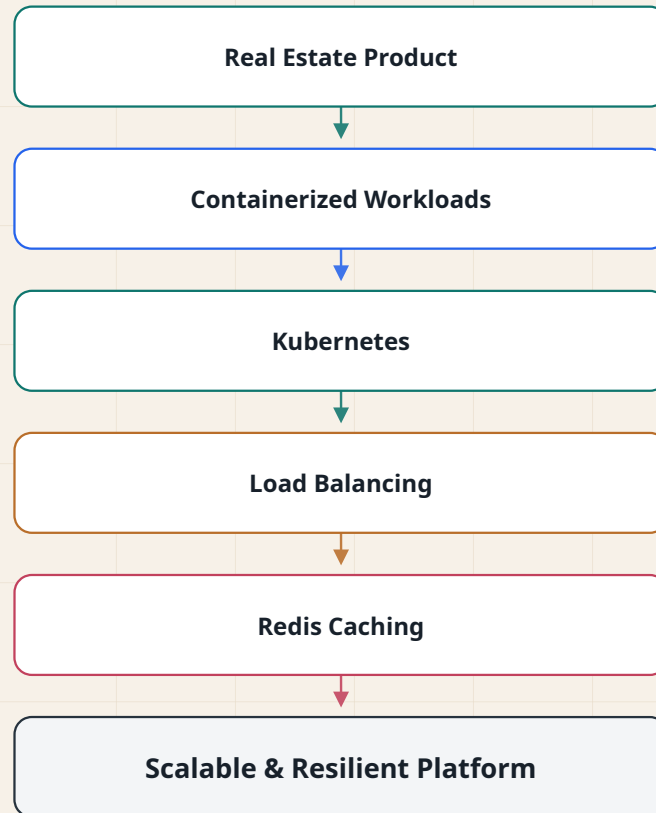
Engineering ownership across infrastructure architecture, deployment, scaling and production troubleshooting.



Product infrastructure engineering - not just server setup.

This Wasn't About Using Kubernetes.

It was about designing infrastructure that could grow with the product.



The engineering question changes when you stop asking:

Does the application work?

and start asking:

What happens when traffic increases, a server fails, or thousands of users request the same data?

Kubernetes Docker Redis Ingress Load Balancing CI/CD Linux Cloud Infrastructure APIs

Muhammad Moez Khan
Full Stack / DevOps / Platform Engineering
Dubai, UAE